Ktx-65 式微处理器汇编语言规范文件

河童重工业会社

平成三十年八月

文件目录

处理器体系架构简介 3

汇编语言规范 4

处理器体系架构简介

处理器可最多访问 64 兆字节，即 16 兆大小的 32 位整形数组，的内存。内存在开始时均会设置为零。

处理器中有十二个寄存器，名称为：R1 至 R4、E1 至 E4、Flag、Val、Ret 和 Line。它们均存放 32 位整形数，开始时默认为零，其中 Flag 寄存器指示于放置有关判断条件的值，Val 寄存器指示于放置数值运算的结果，Ret 寄存器指示于放置函数返回值。

Line 寄存器用于放置当前函数开始的行数，JMP、JIF 命令会以该寄存器为偏移跳转，CALL 命令则不会。

处理器内置了一个数值栈，称为 sAddr 栈，大小为 4 兆字节，可存放 512 千个数字。这个栈用于维护调用时的跳转地址，程序无法直接访问与修改。这个栈的大小使得程序递归不得超过 512 千层，否则会造成栈溢出。

处理器有 30 个可用指令，对于每一个指令可以指定任意多的变量或常量作为参数，但是当参数数量不足时会发生运行时保护，中止当前操作并引发中断。

汇编语言规范

1. 注释格式：[注释内容]

   代表注释的方括号必须两两配对，没有右方括号与之配对的左方括号会将接下来的所有内容记作注释，没有左方括号配对的右方括号会被忽略。

2. 命令格式：<命令名称> [变量...]；其中，变量有：

   常量 %寄存器名称 @固定位置的内存 @%在寄存器值位置的内存 #编译时常量

   编译时常量有：Line（指示当前行数）

   命令对于常量的修改没有任何作用。当寄存器名称错误时会发生编译错误。当变量数量不足时会发生运行时错误。当语句没有以分号结尾时会发生编译错误。

3. 命令列表（其中，<数值>表示第数值个参数）：

特殊命令

指令名称	西文名称	描述
FUNCTION	Function	定义一个函数起始点

调用格式：FUNCTION $函数名称

这等价于在此处放置一条 SET #Line %Line 命令，然后记录这个函数名称所在的行号。这条语句在最终执行时不会体现出来。

CALLFUNC	CallFunction	调用一个定义了的函数

调用格式：CALLFUNC $函数名称

这等价于在此处放置一条 CALL <定值>命令，其中定值为该函数名称所对应的记录的行号。这条语句在运行时也不会体现出来。

控制操作

注：JMP、JIF 命令会以%Line 寄存器为偏移跳转（跳转到相加位置），CALL 命令则不会。

0	UDEF	Undefined	未定义
1	HLT	Halt	停机
2	NOP	NoOp	空指令
3	SET	Set	将<1>的值设为<0>
4	JMP	Jump	跳转至第<0>行语句

编号从文件绝对的最前端开始计数，从一开始。

如果一行有多条语句，则跳转至该行的第一条语句。

5	JIF	JumpIf	跳转至第<0>行语句，如果：

1> 有<1>这个参数，并且<1>为真（不为零），或者：

2> 没有<1>这个参数，并且%Flag 为真

6 CALL Call 调用

将下一行语句的序号与当前%Line 寄存器的值压入 sAddr 栈中，然后跳转至第<0>行语句

7 RET Return 调用返回

将 sAddr 栈顶的下一条语句序号和%Line 寄存器值取出，并且跳转到那一条语句，如果有<0>参数则会将这个参数的值放入%Ret 寄存器。

算术操作

注：算术操作的结果将会放入第<操作数加一>个参数，如果不存在则会放入%Val 寄存器。

8 INV Inverse 取<0>的相反数

9 ADD Add 计算<0>与<1>的和

10 SUB Minus 计算<0>减去<1>的值

11 MULT Multiply 计算<0>与<1>的乘积

12 IDIV IntDivide 计算<0>除以<1>下取整的值

13 MOD Modulo 计算<0>除以<1>所得的余数

14 LSFT LeftShift 计算<0>左移<1>位的值

15 RSFT RightShift 计算<0>右移<1>位的值

16 BAND BitAnd 计算<0>按位与<1>的值

17 BOR BitOr, 计算<0>按位或<1>的值

18 BXOR BitXor 计算<0>按位异或<1>的值

逻辑操作

注：逻辑结果中真为 1，假为 0，结果将会放入<2>参数，如果不存在则会放入%Flag 寄存器。

19 LGR Greater 比较<0>是否大于<1>

20 LLS Less 比较<0>是否小于<1>

21 LGE GreaterEqual 比较<0>是否大于等于<1>

22 LLE LessEqual 比较<0>是否小于等于<1>

23 LEQL Equal 比较<0>是否与<1>相等

24 LAND LogicalAnd 计算<0>和<1>的逻辑与值

25 LOR LogicalOr 计算<0>和<1>的逻辑或值

输入/输出操作

注：如果操作数<0>不存在则以%Val 寄存器作为操作数

26 RINT ReadInt 从控制台输入一个整数并存入<0>

27 RCH ReadChar 从控制台输入一个字符并存入<0>

28 WINT WriteInt 将<0>的数值输出到控制台

29 WCH WriteChar 将<0>这个字符输出到控制台